

For the table:

[https://www.w3schools.com/HTML/html\\_tables.asp](https://www.w3schools.com/HTML/html_tables.asp)

For the CSS:

[https://developer.mozilla.org/en-US/docs/Web/CSS/Guides/Box\\_sizing/Aspect\\_ratios](https://developer.mozilla.org/en-US/docs/Web/CSS/Guides/Box_sizing/Aspect_ratios)

For the canvas:[https://developer.mozilla.org/en-US/docs/Web/API/Canvas\\_API](https://developer.mozilla.org/en-US/docs/Web/API/Canvas_API)

Use of prompts:

Prompt for readme that I used:

"Help me to fix grammar and spelling and help me to format it better and make it clearer without changing anything for my readme this is the Mill River Visual Analytics project Paradise Pond, Smith College, Northampton, MA, 2019 – 2024, This system supports the long-term monitoring of the Mill River upstream and downstream of Paradise Pond. The data tracks macroinvertebrate communities — small aquatic organisms that serve as biological indicators of river health — collected each summer and fall from 2019 to 2024. The goal is to help researchers understand how environmental changes such as conductivity from road salt, water temperature, and stream discharge affect which organisms are present and in what densities. we worked with researcher: Marney Pratt, Biology Department, Smith College in the dataset: 238 species, 84 families, Upstream and Downstream locations during Summer and Fall seasons for 2019-2023 Running the System so the system is loading local JSON data files this means it needs to be served over a local HTTP server Install Python then open terminal and go to the project folder then start a local server with `-m http.server 8000` and then open your browser and go to `__http://localhost:8000__` Navigating the System on the top there are three buttons you can use them to switch between the views so let me explain each one.

So Communities Across Seasons is showing 10 scatterplots side-by-side they are organized by year which is 2019 to 2023 and season where Summer is on left and Fall is on right. Each plot is showing Water Temperature on x-axis and Log scale of Density of macroinvertebrates on y-axis and the points are colored by their functional feeding group. For use the brush tool you need go on the Summer 2019 plot and click and drag to make a rectangle around the dots you want to select. The dots that are matching will highlight with red outline on all the other plots which is useful to track if the same organisms are appearing at similar densities and temperatures in different seasons and years. On the left side of each row you can see the year label and also there is a legend at top that shows you the color mapping for feeding groups. For Environmental Effects you need use the dropdowns on left side to select an environmental variable and a functional feeding group then two scatterplots will appear side by side which is Upstream on left and Downstream on right. Each point represents one organism observation and density is shown in log scale and points are colored by functional feeding group. Both plots are sharing the same axis scale so you can do direct comparison of upstream vs downstream which is quite helpful. Anomalous Organisms is showing a correlation matrix of organism families vs environmental variables and the cells that are colored it means there is a notable correlation where blue is positive and red is negative. You can check the button that says "Show anomalies only" for filter to rows where  $|r| > 0.3$  and if you click on any colored cell then on the right side will appear a detailed scatter plot that is showing the density of that organism vs the environmental variable you selected. Known Bugs so the brush interaction in Communities

Across Seasons is only listening for selections that you make on the Summer 2019 plot which is top left and if you try brushing on other plots it will not trigger . Also when you first load the page the Environmental Effects view is showing blank plots and it will stay like that until you select both of the dropdowns. Another thing is the scatter plot in Anomalous Organisms is using linear scale so if there is extreme density outliers then most of points will be compressed to the bottom which make it hard to see. Data Cleaning and Analysis so the raw data was collected by Marney Pratt's lab from field samples that were taken upstream and downstream of Paradise Pond each summer and fall from 2019 to 2023. The records that had missing or non-finite values for environmental variables or density they were excluded from the visualizations and only the observations where density > 0 were included in scatterplots. Also family and feeding group labels were standardized to have consistent casing for the matching across files. The merged\_data.json is the cleaned and merged observation records and it includes organism, family, feeding group, location, season, year, density, and environmental measurements. The correlations.json it has the Pearson correlation coefficients that were computed between each organism family density and each environmental variable calculated across all available observations. so D3.js v7 is included locally as d3.v7.min.js and there is no other external libraries that are needed. Folder has: index.html which is the main HTML, styles.css for the styling, script.js for all the visualization logic, d3.v7.min.js which is the D3.js library, merged\_data.json which is the cleaned data, correlations.json for the correlations, and README.md which is this file."

Screenshot of zoomed-out whole prompt:

Help me to fix grammar and spelling and help me to format it better and make it clearer without changing anything for my readme this is the Mill River Visual Analytics project Paradise Pond,Smith College, Northampton, MA, 2019 - 2024. This system supports the long-term monitoring of the Mill River upstream and downstream of Paradise Pond. The data tracks macroinvertebrate communities — small aquatic organisms that serve as biological indicators of river health — collected each summer and fall from 2019 to 2024. The goal is to help researchers understand how environmental changes such as conductivity from road salt, water temperature, and stream discharge affect which organisms are present and in what densities. we worked with researcher: Marney Pratt, Biology Department, Smith College in the dataset: 238 species, 84 families, Upstream and Downstream locations during Summer and Fall seasons for 2019-2023 Running the System so the system is loading local JSON data files this mean it need to be served over a local HTTP server install Python then open terminal and go to the project folder then start a local server with -m http.server 8000 and then open you browser and go to <http://localhost:8000> . Navigating the System on the top there are three buttons you can use them for switch between the views so let me explain each one.

So Communities Across Seasons is showing 10 scatterplots side-by-side they are organised by year which is 2019 to 2023 and season where Summer is on left and Fall is on right. Each plot is showing Water Temperature on x-axis and Log scale of Density of macroinvertebrates on y-axis and the points are colored by their functional feeding group. For use the brush tool you need go on the Summer 2019 plot and click and drag for make a rectangle around the dots you want to select. The dots that are matching will highlight with red outline on all the other plots which is useful for track if the same organisms are appearing at similar densities and temperatures in different seasons and years. On the left side of each row you can see the year label and also there is a legend at top that show you the color mapping for feeding groups.

For Environmental Effects you need use the dropdowns on left side for select a environmental variable and a functional feeding group then two scatterplots will appear side by side which is Upstream on left and Downstream on right. Each point is represent one organism observation and density is shown in log scale and points are colored by functional feeding group. Both plots are sharing the same axis scale so you can do direct comparison of upstream vs downstream which is quite helpful.

Anomalous Organisms is showing a correlation matrix of organism families vs environmental variables and the cells that are colored it means there is a notable correlation where blue is positive and red is negative. You can check the button that say "Show anomalies only" for filter to rows where  $|r| > 0.3$  and if you click on any colored cell then on the right side will appear a detailed scatter plot that is showing the density of that organism vs the environmental variable you selected.

Known Bugs so the brush interaction in Communities Across Seasons is only listening for selections that you make on the Summer 2019 plot which is top left and if you try brushing on other plots it will not trigger . Also when you first load the page the Environmental Effects view is showing blank plots and it will stay like that until you select both of the dropdowns. Another thing is the scatter plot in Anomalous Organisms is using linear scale so if there is extreme density outliers then most of points will be compressed to the bottom which make it hard to see.

Data Cleaning and Analysis so the raw data was collected by Marney Pratt's lab from field samples that were taken upstream and downstream of Paradise Pond each summer and fall from 2019 to 2023. The records that had missing or non-finite values for environmental variables or density they were excluded from the visualizations and only the observations where density  $> 0$  were included in scatterplots. Also family and feeding group labels were standardized to have consistent casing for the matching across files. The merged\_data.json is the cleaned and merged observation records and it includes organism, family, feeding group, location, season, year, density, and environmental measurements. The correlations.json it has the Pearson correlation coefficients that were computed between each organism family density and each environmental variable calculated across all available observations.

so D3.js v7 is included locally as d3.v7.min.js and there is no other external libraries that are needed. Folder has: index.html which is the main HTML, styles.css for the styling, script.js for all the visualization logic, d3.v7.min.js which is the D3.js library, merged\_data.json which is the cleaned data, correlations.json for the correlations, and README.md which is this file.

Show Docs

Here's the updated README based on everything I now know about your final system:

## Mill River Visual Analytics

Paradise Pond · Smith College · Northampton, MA 2019 - 2024

### Purpose

This visual analytics system supports long-term biological monitoring of the Mill River upstream and downstream of Paradise Pond at Smith College. The data tracks macroinvertebrate communities — small aq. organisms that serve as biological indicators of river health — collected each summer and fall from 2019 to 2024.

Here's the updated README based on everything I now know about your final system:

---

## Mill River Visual Analytics

Paradise Pond · Smith College · Northampton, MA 2019 - 2024

---

### Purpose

This visual analytics system supports long-term biological monitoring of the Mill River upstream and downstream of Paradise Pond at Smith College. The data tracks macroinvertebrate communities — small aquatic organisms that serve as biological indicators of river health — collected each summer and fall from 2019 to 2024.

The system is designed to help researchers understand:

- How macroinvertebrate communities and functional feeding groups change across seasons and years
- How environmental variables (conductivity, water temperature, stream discharge, etc.) relate to organism density
- Which organisms show unusual or anomalous responses to environmental conditions

**Researcher:** Marney Pratt, Biology Department, Smith College **Dataset:** 238 species · 84 families · Upstream & Downstream locations · Summer & Fall seasons · 2019-2023

---

### How to Use

#### Running the System

Because the system loads local JSON data files, it must be served over a local HTTP server — opening the HTML file directly in a browser will not work.

1. Make sure Python is installed
2. Open a terminal and navigate to the project folder:

```
bash
cd /path/to/project
```

3. Start a local server:

```
bash
python3 -m http.server 8080
```

4. Open your browser and go to <http://localhost:8080>
- 

### Navigating the System

Use the three buttons at the top to switch between views.

#### 1. Communities Across Seasons

Shows 10 side-by-side scatterplots organized by year (2019-2023) and season (Summer left, Fall right). Each plot shows Water Temperature (x-axis) vs Log scale of Density of macroinvertebrates (y-axis). Points are colored by functional feeding group.

- Use the **brush tool** on the Summer 2019 plot — click and drag to draw a rectangle around dots
- Matching dots will highlight with a red outline across all other plots
- This lets you track whether the same organisms appear at similar densities and temperatures across different seasons and years
- Each row is labeled with the year on the left side
- A legend at the top shows the feeding group color mapping

#### 2. Environmental Effects

- Use the dropdowns on the left to select an environmental variable and a functional feeding group
- Two scatterplots appear side by side — Upstream (left) and Downstream (right)
- Each point represents an organism observation; density is shown on a log scale
- Points are colored by functional feeding group
- The shared axis scale between the two plots allows direct comparison of upstream vs downstream patterns

## 2. Environmental Effects

- Use the dropdowns on the left to select an environmental variable and a functional feeding group
- Two scatterplots appear side by side — Upstream (left) and Downstream (right)
- Each point represents an organism observation; density is shown on a log scale
- Points are colored by functional feeding group
- The shared axis scale between the two plots allows direct comparison of upstream vs downstream patterns

## 3. Anomalous Organisms

- Shows a correlation matrix of organism families vs. environmental variables
- Colored cells indicate a notable correlation (blue = positive, red = negative)
- Check "Show anomalies only" to filter to rows with  $|r| > 0.3$
- Click any colored cell to see a detailed scatter plot on the right showing that organism's density vs the selected environmental variable

---

## Known Bugs

- The brush interaction in **Communities Across Seasons** only listens for selections made on the Summer 2019 (top left) plot — brushing on other plots will not trigger highlights
- On first load, the **Environmental Effects** view shows blank plots until both dropdowns are selected
- The detail scatter plot in **Anomalous Organisms** uses a linear scale; extreme density outliers can compress most points toward the bottom

---

## Data Cleaning & Analysis

### Source Data

Raw data was collected by Marney Pratt's lab from field samples taken upstream and downstream of Paradise Pond each summer and fall, 2019–2023.

### Cleaning Steps

- Records with missing or non-finite values for environmental variables or density were excluded from visualizations
- Only observations with density  $> 0$  were included in the scatterplots
- Family and feeding group labels were standardized to consistent casing for matching across files

### Derived Data

- `merged_data.json` — cleaned and merged observation records including organism, family, feeding group, location, season, year, density, and environmental measurements
- `correlations.json` — Pearson correlation coefficients computed between each organism family's density and each environmental variable, calculated across all available observations

---

## Dependencies

- D3.js v7 (included locally as `d3.v7.min.js`)
- No other external libraries required

---

## File Structure

```
project/
├─ index.html           # Main HTML structure
├─ styles.css           # All styling
├─ script.js            # All visualization and interaction logic
├─ d3.v7.min.js         # D3.js library (local copy)
├─ merged_data.json     # Cleaned observation data
├─ correlations.json    # Precomputed correlation coefficients
└─ README.md           # This file
```



The time that shows when I hover on the prompt is below:

missing or non finite values for environmental variables or density they were excluded from the visualizations and only the observations where density > 0 were included in scatterplots. Also family and feeding group labels were standardized to have consistent casing for the matching across files. The merged\_data.json is the cleaned and merged observation records and it includes organism, family, feeding group, location, season, year, density, and environmental measurements. The correlations.json it has the Pearson correlation coefficients that were computed between each organism family density and each environmental variable calculated across all available observations. so D3.js v7 is included locally as d3.v7.min.js and there is no other external libraries that are needed. Folder has: index.html which is the main HTML, styles.css for the styling, script.js for all the visualization logic, d3.v7.min.js which is the D3.js library, merged\_data.json which is the cleaned data, correlations.json for the correlations, and README.md which is this file.

Show less

Apr 30   

Here's the updated README based on everything I now know about your final system:

## Mill River Visual Analytics

Paradise Pond · Smith College · Northampton, MA 2019 – 2024



For GitHub issues:

```
so my teammates updated the repo one forked it and it says now i need to
aslo make changes so what do i do This branch is [1 commit ahead of]
(https://github.com/angelx34/Mill-River-Visual-Analytics/compare/adrianasoldat%3AMill-River-Visual-
Analytics%3Amain...main) and [10 commits behind]
(https://github.com/angelx34/Mill-River-Visual-Analytics/compare/main...adrianasoldat%3AMill-River-Visual-
Analytics%3Amain) adrianasoldat/Mill-River-Visual-Analytics:main.
```

Your fork is out of sync with the original repo. Here's what to do:

### Sync your fork with the upstream repo

1. Add the upstream remote (only need to do this once):

```
bash
git remote add upstream https://github.com/adrianasoldat/Mill-River-Visual-Analytics
```

2. Fetch the latest changes from upstream:

```
bash
git fetch upstream
```

3. Merge upstream changes into your branch:

```
bash
git merge upstream/main
```

4. Push the updated code to your fork:

```
bash
git push origin main
```

### What the message means

- **1 commit ahead** → You have 1 commit that the original repo doesn't have yet (your changes)
- **10 commits behind** → The original repo has 10 new commits you haven't pulled in yet

### If you get merge conflicts

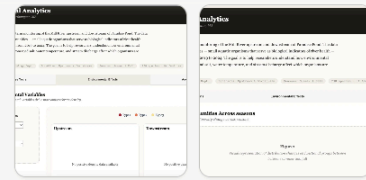
When you run `git merge`, Git might flag files that both you and your teammates edited. For each conflict you'll see something like:

```
<<<<<< HEAD
your changes
=====
their changes
>>>>>> upstream/main
```

Just open those files, pick which version to keep (or combine both), then:

```
bash
git add .
git commit -m "Merge upstream changes"
git push origin main
```

Once synced, you can open a **Pull Request** from your fork to the main repo to contribute your 1 commit back!



this is my friend fork and this is my original and original is broken for some reason

I can see the difference:

- **Your repo (adrianasoldat)** = broken, charts not showing, empty figure placeholder
- **Teammate's fork (angelx34)** = working, charts showing with data

So your teammate actually has a **better working version** than yours. You need to pull their specific working code. Let's do a fresh clone of their repo:

```
bash
cd ~/Documents/csc\ 235/
git clone https://github.com/angelx34/Mill-River-Visual-Analytics.git WorkingVersion
cd WorkingVersion
python3 -m http.server 8080
```

Then go to <http://localhost:8080> and see if it looks like your teammate's version. If it does, **this is the folder you should work from going forward.**



For a general and deeper understanding of the concepts:

im using canvas for scatter plot and i do clearrect before every redraw, i get that without it old dots stay but i want to understand deeper, like does canvas have any memory of what drew the dots or is it literally just pixels, and can i ever remove just one dot without redrawing everything

Great question. Here's the deep answer:

---

**Canvas has zero memory — it's literally just pixels.**

Once you call `ctx.arc()` and `ctx.fill()`, the canvas has no idea a "dot" was drawn. It just set some pixels to a color. There is no object, no reference, no metadata — just a grid of colored pixels. The canvas cannot tell the difference between a dot you drew and a pixel that was already that color.

---

**So when you call `clearRect` you're not "removing dots" — you're painting a rectangle of transparent pixels over everything.** It's the digital equivalent of whiting out a piece of paper. The previous drawing is just gone.

---

**Can you ever remove just one dot without redrawing everything?**

Technically yes but practically no — your options are:

1. **Redraw everything minus that dot** — which is what everyone does, and what you're doing
2. **Paint over just that dot with the background color** — but this breaks if dots overlap, you'd erase part of another dot
3. **Use a separate canvas layered on top** — draw the dot you want to "remove" on the top canvas, then just clear that top canvas. This is how games handle it.

## HTML Table vs D3/SVG

These are fundamentally different tools for displaying tabular/visual data:

**HTML** `<table>` is a semantic document element. It's designed for structured, tabular data — rows and columns of text/numbers. The browser handles all layout. It's great for static or lightly interactive data grids, accessibility (screen readers understand it natively), and is trivial to style with CSS.

**D3 + SVG** is a programmatic drawing system. D3 manipulates SVG elements (rectangles, circles, paths, axes) based on data. Nothing is "table-like" by default — you construct every visual element yourself. This gives you complete control over layout, animation, transitions, and custom encodings (color, size, position).

The core tradeoff:

|                      | HTML Table                   | D3/SVG                                |
|----------------------|------------------------------|---------------------------------------|
| <b>Purpose</b>       | Display rows/columns of data | Visualize data as graphics            |
| <b>Layout</b>        | Browser-managed              | You control every pixel               |
| <b>Data binding</b>  | Manual DOM updates           | Data-driven joins (enter/update/exit) |
| <b>Animation</b>     | CSS transitions (limited)    | First-class, smooth transitions       |
| <b>Accessibility</b> | Excellent (native semantics) | Requires extra ARIA work              |
| <b>Complexity</b>    | Low                          | High                                  |
| <b>Use case</b>      | Spreadsheet-like data        | Charts, graphs, custom visuals        |

**When to use a table:** You have rows of records a user needs to read, sort, or scan — like a leaderboard, invoice, or data export.

**When to use D3/SVG:** You want to encode data visually — bar charts, scatter plots, network graphs, geographic maps, or any custom visualization where *position, size, or color* carries meaning.

A common pattern is to use **both together** — a D3 chart for the visual summary, and an HTML table below it for the raw data.

so im working on the data loading part, first one is about promise.all i used it to load both json files at same time and i know its faster but i dont really understand whats actually happening differently, like is it the network that changes or is it javascript doing something different? also when would you not want to use promise.all even if the files dont depend on each other. also so i changed my correlations data from array to object using object.fromentries so i can look up organisms by name faster, i know objects are faster for lookup but explain why exactly and about try catch i wrapped my fetch calls in it to handle when someone opens the file without a server but im not sure what exactly throws the error in that situation, is it the fetch itself or the json parsing after, and also whats the difference between doing it this way vs just checking response.

Great questions, here's the deep answer on all four:

---

### **Promise.all — what's actually changing**

It's the **network** that changes, not JavaScript. JavaScript is still single-threaded — it can't literally do two things at once. What Promise.all does is fire both fetch requests almost simultaneously without waiting for the first to finish before starting the second. So if each file takes 200ms to load, sequential fetching takes 400ms total, Promise.all takes ~200ms because both requests are in flight at the same time. The browser handles the actual network requests in parallel in the background, JavaScript just coordinates when to continue once both are done.

Great questions, here's the deep answer on all four:

---

### Promise.all — what's actually changing

It's the **network** that changes, not JavaScript. JavaScript is still single-threaded — it can't literally do two things at once. What Promise.all does is fire both fetch requests almost simultaneously without waiting for the first to finish before starting the second. So if each file takes 200ms to load, sequential fetching takes 400ms total, Promise.all takes ~200ms because both requests are in flight at the same time. The browser handles the actual network requests in parallel in the background, JavaScript just coordinates when to continue once both are done.

---

### When would you NOT want Promise.all

- **When one request depends on the other** — if you need data from file 1 to know what to fetch from file 2, you can't parallelize that
  - **When you want partial results** — Promise.all is all or nothing. If one fails, the whole thing fails and you get nothing. If you'd rather show partial data when one file loads, use `Promise.allSettled` instead which resolves even if some fail
  - **When one file is huge** — if one file is 10mb and one is 1kb, the small one finishes fast but you're still waiting on the big one anyway, so the parallelization benefit shrinks
- 

### Why object lookup is faster than array lookup

When you search an array for an organism by name, JavaScript has to check every element one by one until it finds a match — that's  $O(n)$ , meaning if you have 500 organisms it might check all 500.

When you access an object by key like `correlationsData[organism]`, JavaScript computes a **hash** of the key string and goes directly to that memory location — that's  $O(1)$ , meaning it doesn't matter if you have 5 or 5000 organisms, it's the same speed. It's like the difference between scanning every page of a book vs going straight to the index.

`Object.fromEntries` is just converting your array into that indexed structure in one line:

```
javascript

// Before - array, slow lookup
corrArray.find(row => row.organism === name)

// After - object, instant lookup
correlationsData[name]
```

---

### What actually throws the error in your try/catch

It's the **fetch itself** that throws when there's no server. When you open the HTML file directly (`file://` protocol), the browser blocks the fetch request entirely for security reasons — it never even attempts the network request. So `fetch()` rejects its promise immediately and that rejection is what your catch block catches.

The JSON parsing (`r.json()`) would throw a different kind of error — it throws if the response comes back but the content isn't valid JSON. That's a separate failure mode.

So you actually have two different errors your catch handles without distinguishing between them:

- No server → fetch rejects
- Bad JSON → `.json()` throws

